

Welcome to coding at SARP 2026!

Lesson 2: Tabular Data with pandas

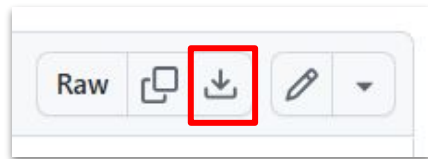
As you arrive and get set up...



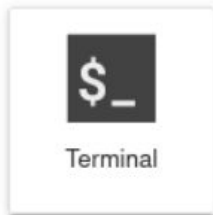
1. Open **CryoCloud**: hub.cryointhecloud.com
 - Start a 1.9GB Python node
 - Create a folder named `lessons` (or any way you want to organize it)

2. Make a copy of **today's lesson notebook**:

- In your web browser, visit:
https://github.com/NASA-SARP/2026-Coding-Alex/blob/main/lessons/02_tabular_data.zip
- Download the zip folder `02_tabular_data.zip` and unzip:
- “Drag and drop” the unzipped folder `02_tabular_data` into the `lessons` folder you created earlier in CryoCloud



[2. *Advanced option*]*—*use the terminal to copy the folder in CryoCloud instead:



```
cp -ri ~/shared-public/NASA-SARP/2026-Coding-Alex/lessons/02_tabular_data  
~/Projects/lessons
```

A reminder before we start...

- At times, this may (should) feel **challenging...** we (including your peers) are here to help you!
- Don't worry if you don't pick up everything we teach in the first 2 weeks
- Your **coding skills do not make you a good or bad scientist** (however, coding can sometimes help us turn research ideas into results more quickly and efficiently...)
- Remember to **breathe + enjoy the experience:** reflect on what you are learning, how the journey of learning feels, and the forms of learning that work best for you

Coding lesson philosophy



Active learning

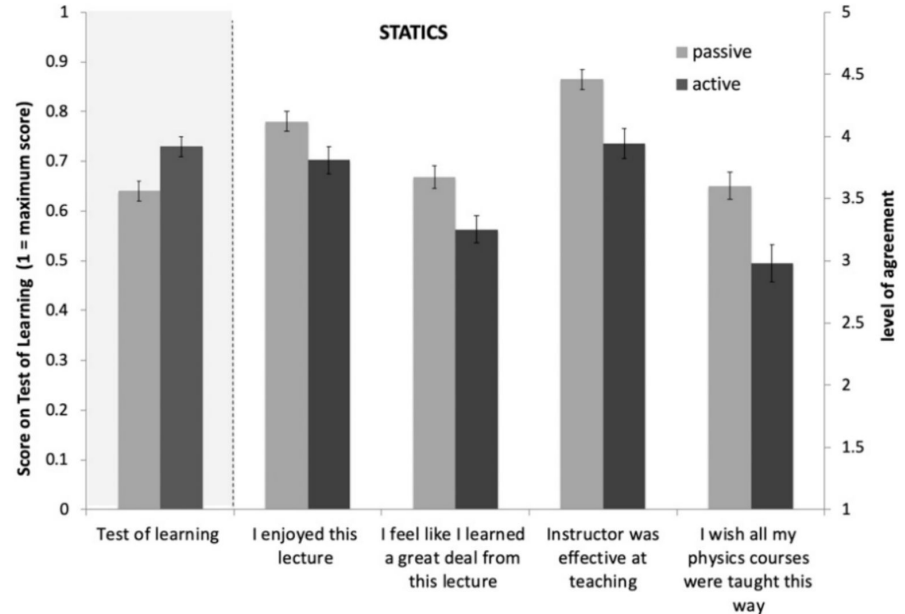
Practice
problems, group
work, clicker
questions



Passive learning

Lecturing

Why active learning...?



Deslauriers et al. (2016) Proc. Nat. Acad. Sci.

Recap: Goals for our coding lessons

- **Build a conscious mental framework for how code is structured**—so that you can notice patterns and code organization beyond syntax
- **Be more comfortable tackling unknown problems**—however skilled you are, the coding journey always involves learning
- **Grow computational skills beyond code writing**—computers, data formats, and other skills that make for a well-rounded programmer

Recap: Goals for our coding lessons

Data types



Data structures



Keeping track of individual variables (pieces of data) would get very out of hand very quickly → to organize our data in coding we build **data structures**

Recap: From data types to data structures

Data types

Data type	Python name	What it holds	Examples
Integer	<code>int</code>	Whole numbers	<code>14</code> , <code>0</code> , <code>-3</code>
Float	<code>float</code>	Decimal numbers	<code>3.5</code> , <code>-0.2</code> , <code>6.022e23</code>
String	<code>str</code>	Text	<code>"Jack"</code> , <code>'hello'</code>
Boolean	<code>bool</code>	True or False	<code>True</code> , <code>False</code>

Data structures

List

```
pollutants = ['co', 'co2', 'no2', 'o3']  
ph_values = [4.5, 1.0, 4.3, 4.3, 7.6]  
mixed = ['abc', 34, True, 40, 'male']
```

Dictionary

```
metadata = {  
    'location': 'Nairobi',  
    'quality_flag': 5,  
    'zenith': 60,  
    'clouds': True,  
}
```

DataFrame

	temperature	humidity	wind_speed
0	25	60	10
1	20	65	9
2	27	62	10
3	31	80	6

Anatomy of a `pd.DataFrame`

`df =`

	temperature	humidity	wind_speed
0	25	60	10
1	20	65	9
2	27	62	10
3	31	80	6

← Column
names

← A single
row

↑
Indexes

↑
A single column—a `pd.Series`

Accessing values in a `pd.DataFrame`

	temperature	humidity	wind_speed
0	25	60	10
1	20	65	9
2	27	62	10
3	31	80	6

analyst	temperature	humidity	wind_speed
Johan	25	60	10
Priya	20	65	9
Ángel	27	62	10
Martha	31	80	6

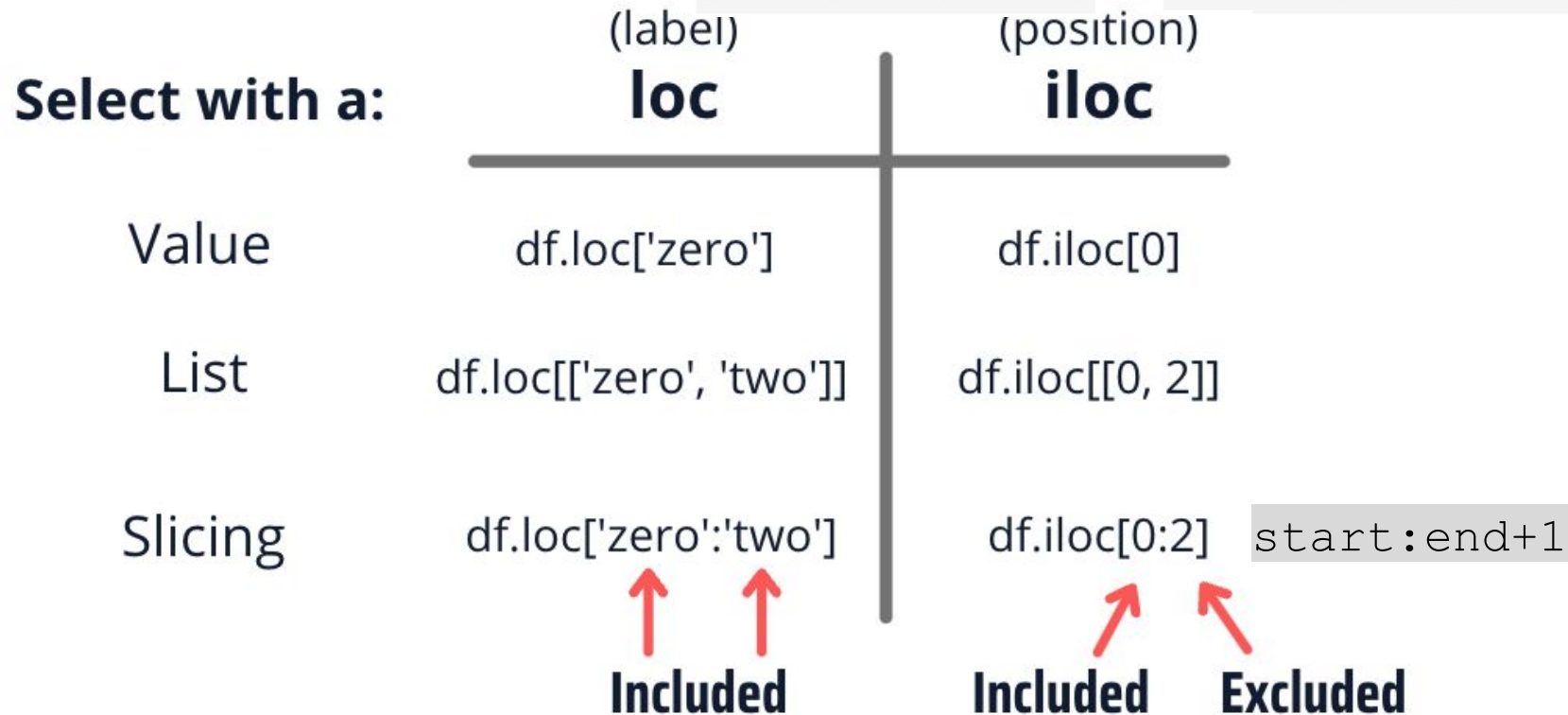
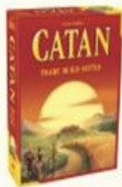


Figure: The PyCoach via Medium

Importing libraries

Library: additional functionality (ex. data structures or functions) that you can make available to your code by importing them

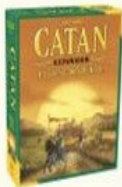
```
import os
import datetime
import pandas as pd
import xarray as xr
```



Base Game



Seafarers



Cities & Knights



Traders & Barbarians



Explorers & Pirates

Debugging

Traceback (most recent call last):

File `"/path/to/example.py"`, line 4, in `<module>`

`greet('Chad')`

...

File `"/path/to/example.py"`, line 2, in `greet`

`print('Hello, ' + someon)`

`NameError: name 'someon' is not defined`

read from bottom to top

Debugging

Error type

When it occurs

`NameError`

trying to use a variable that doesn't exist

`IndexError`

trying to access a value using an index and that index does not exist in the object

`SyntaxError`

something that breaks Python grammar rules (e.g., extra parenthesis, mismatching quotes, backwards brackets...)

`TypeError`

trying to do something to an object (e.g., a function, method, or operator) that the object can't handle

`AttributeError`

trying to get information from an object and the type of information doesn't exist on that type of object

`ValueError`

when a function/method gets an input value that has the right type but the value is not appropriate

Debugging

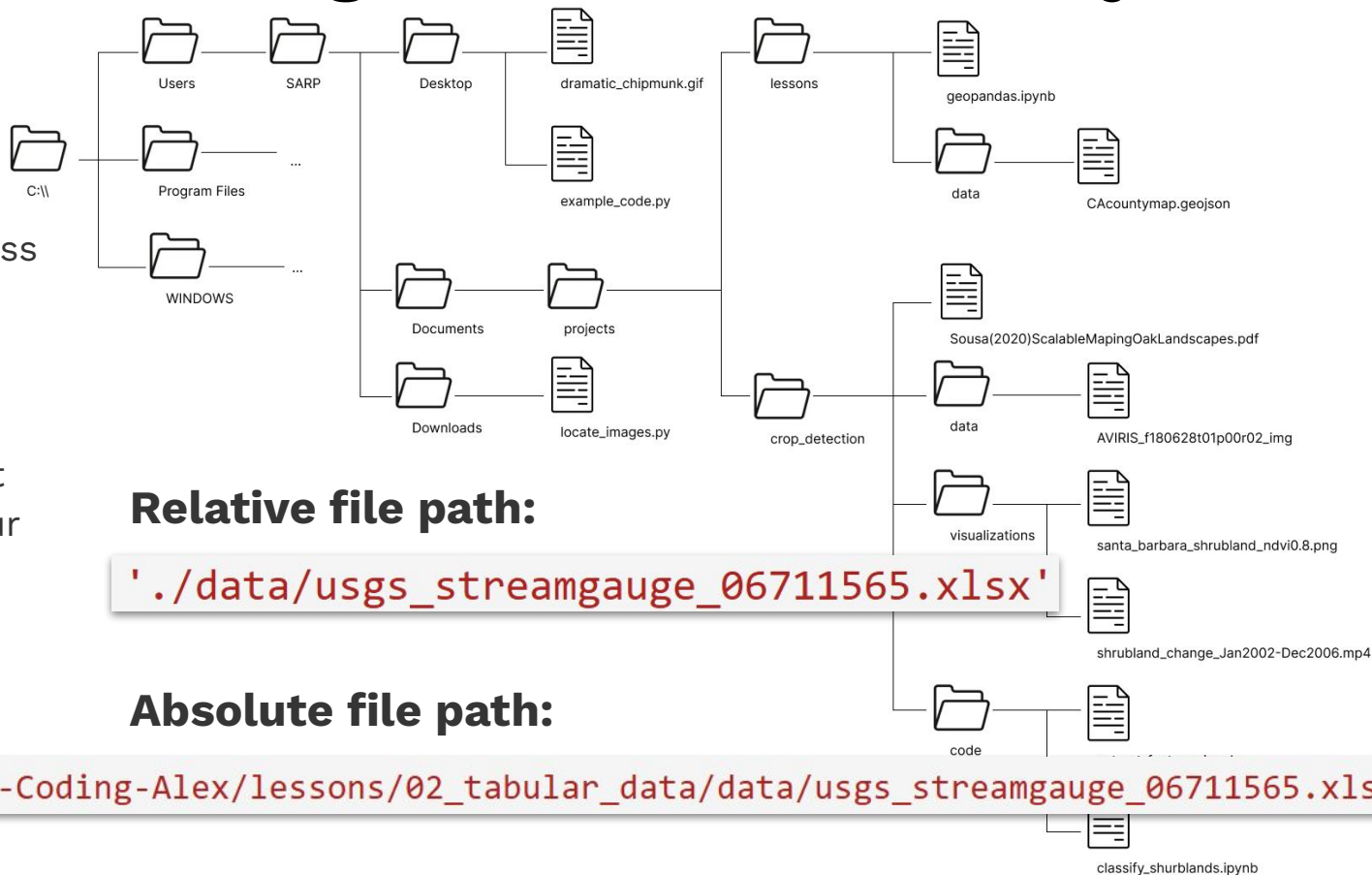


Debugging diary

Why not start a debugging diary to record all the bugs you encounter over your SARP coding journey?
You could include:

- Types of bug and why you think it happened
- The steps you took to identify and locate it
- What you did to fix it
- How it made you feel (before, during, and after the bug)
- Motivational words to your future self the next time you debug!

Computers are organized “hierarchically”



Filepath: An address that specifies the location of any file on your computer

Root: The folder at the very top of your tree that contains everything

Relative file path:

`'./data/usgs_streamgauge_06711565.xlsx'`

Absolute file path:

`'/Projects/2026-Coding-Alex/lessons/02_tabular_data/data/usgs_streamgauge_06711565.xlsx'`

Lesson 2: Tabular Data with pandas

Lesson 2: Tabular Data with pandas

Exit ticket <https://forms.gle/ixiPEYj7Tmo61Q1Q7>

Don't forget to shut down CryoCloud:
File → Hub Control Panel → Stop Server

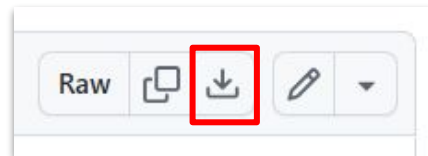
As you arrive and get set up...

1. Open **JupyterLab** on your SARP laptop:
 - Hit the **Jupyter Lab launcher** on your desktop



2. Make a copy of **today's lesson notebook**:

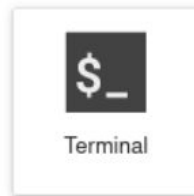
- In your web browser, visit:
https://github.com/NASA-SARP/2026-Coding-Alex/blob/main/lessons/02_tabular_data.zip
- Download the zip folder `02_tabular_data.zip` and unzip:
- “Drag and drop” the unzipped folder `02_tabular_data` into Jupyter Lab



[2. Advanced option]—use the terminal to copy the folder instead:

```
cp -ri C:\Users\sarp\Downloads\01_building_blocks\01_building_blocks  
C:\Users\sarp\projects\lessons
```

3. Open the terminal and run: `conda install jupyterlab-myst`



Local option